

Embeddable Widget & Lead-Capture Platform

Let a customer define a widget, hand them one line of `<script>`, and safely catch everything the public internet throws back at you — validated, spam-filtered, enriched, and dashboarded.

Difficulty: Medium–Hard

Self-paced · no deadlines

JavaScript or Python

Public GitHub repo

\$0 · no credit card, ever

THE FLAVOR

The public internet is your input:
CORS, caching, abuse, resilience

YOU WILL MASTER

Public API security · CORS · rate
limiting · caching · fallback chains

YOUR \$0 STACK

Node or Python · Docker Postgres ·
2 free geo APIs · plain HTML test
page

How to read this document: § 1–2 tell you what this capstone is and whether it's the right pick for you. § 3–9 are the build: rules, features, architecture, the definition of done, and the build phases. § 10–12 are the practical frame: the free tools, the GitHub rules, and exactly how you'll be evaluated. § 13–14 prepare your demo and give you hand-picked, free resources for every phase. You don't need to memorize it — work through the § 8 phases at your own pace and come back when you need detail.

Contents

1. The mission.....	1	9. Stretch goals	7
2. What it takes to finish.....	2	10. Your \$0 stack	7
3. Ground rules	2	11. GitHub rules.....	8
4. What you'll build.....	3	12. How it's evaluated	9
5. Architecture overview	4	13. The final demo	10
6. Definition of done	5	14. Curated resources.....	11
7. Realistic scope	5	15. Glossary	12
8. The build, phase by phase	6		

1 · The mission

In this capstone you build a platform that lets customers create **embeddable widgets** — signup forms, contact forms, call-to-action popovers — and install them on **any website** with a single `<script>` tag.

When a visitor interacts with the widget, the submission travels back to *your* backend, where it is validated, protected against abuse, **enriched** with location data, stored, and shown to the widget's owner in a dashboard.

Unlike every previous project in the track, **your application will receive requests directly from browsers you don't control**. That single fact changes everything: you can't trust the input, you can't control the traffic, and you can't predict the origin. You'll think like a backend engineer building software for the open internet — because that is exactly what you'll be doing.

This is a real product category. Intercom chat bubbles, Mailchimp signup forms, HubSpot lead popups — all of them are exactly this system: a script snippet, a config endpoint, and a hardened public submission API. FlyRank runs its own version (the popover/CDN widget system); you're building the same machine from scratch, your way.

Feeling intimidated? Don't be. Every piece here is one you've already built in a track assignment — validation, auth, caching, webhooks. The capstone is *assembly and hardening*, not a greenfield leap. And the wow moment is real: the first time your widget appears on a page you didn't build, it stops feeling like homework.

2 · What it takes to finish

Honest picture before you commit — this is a **Medium–Hard** capstone, and here is what actually makes it hard:

The three genuinely hard parts

- **CORS debugging.** The browser blocks cross-origin requests with terse console errors and invisible **preflight** requests. Expect an afternoon of confusion — it's a rite of passage. The MDN guide in § 14 is your map.
- **Thinking like an attacker.** Rate limiting and spam controls only make sense once you imagine someone *deliberately* flooding your endpoint. You'll write tests that attack your own API — a genuinely new skill.
- **Failure that must not fail.** The enrichment chain and the email side effect both practice the same discipline: a broken dependency must degrade the response, never destroy it. Getting the error-handling boundaries right takes design, not just code.

Time budget: plan for roughly **35–50 focused hours**, entirely at your own pace — the phases in § 8 slice it into assignment-sized chunks. If you can only ship the core (§ 6), that is a real, passing capstone.

You already have the parts

Piece of this capstone	Where you already built it
Validation at the boundary, idempotent submissions	A5 / A6
Multi-tenant data model & auth	A3
Webhooks + keeping secrets out of code	A14
Cached config delivery	A19
Security review habits — this is the most-attacked surface in the track	A20

Choose with your curiosity, not the stars. A medium capstone done excellently beats a hard one that falls over. Pick this one if "my code runs on a site I don't control" sounds exciting.

3 · Ground rules — read before you start

These rules are the same for every capstone in the track. They exist so that 20,000+ interns can be evaluated fairly — and so your finished project is something you can safely show in an interview.

The five rules

Rule	What it means for you
Pick one, early	This internship is self-paced — no deadlines. Still, choose your capstone early (once the first assignments have shown you the landscape) and write a one-page design doc (problem, data model, API surface, layer sketch, one explicit non-goal). Phase 1 in § 8 is exactly that doc.
One separate, public repo	The capstone lives in its own public GitHub repository from day one — never inside your assignments repo. Full rules in § 11.
\$0, no credit card — ever	Everything can be built with free tools; this document lists the exact free stack in § 10. If you ever find yourself on a page asking for a credit card, stop — you took a wrong turn, the free path exists.
AI-assisted building is encouraged — and owned	Use AI tools freely, but keep BUILDLLOG.md honest: where AI helped, where it was wrong, what you changed. At the demo you will be asked to explain 2–3 lines the evaluator picks. <i>"The AI wrote it"</i> is not an answer.
Propose your own	You may pitch a different capstone idea against the shared requirements (§ 12). If it exercises a comparable skill set, mentors can approve it.

CONSTRAINTS FOR THIS CAPSTONE

- **No real CDN, domain, or hosting needed.** The "customer site" is just a plain HTML file served from a *different origin* than your API (a second local port, or a `file://` page). Localhost is real programming.
- **Mock the geo providers in tests.** Use the real free APIs while developing, but your fallback test must be deterministic: one mock provider that answers, one you can toggle "down".
- **Never commit an API key or SMTP credential.** Everything secret lives in `.env` (git-ignored) with a committed `.env.example`.

4 · What you'll build

A platform with five moving parts. Build them in this order — each one teaches a named production concern:

1 Widget management API

teaches: multi-tenant CRUD + auth

An **authenticated** admin API where a customer creates and manages widgets: type (signup form / CTA / popover), title and description, form fields, button text, display options. Widgets are **tenant-isolated** — one customer can never see or edit another's widgets. Full CRUD, validated, honest status codes.

2 Embed snippet generation

teaches: developer experience

Once a widget exists, your API returns the one line the customer pastes into any website:

```
<script src="https://your-domain.com/widget.js?id=abc123"></script>
```

That single line is your product's front door. Everything it needs — config, rendering, submission wiring — must flow from it automatically.

3 Fast, cached widget delivery

teaches: HTTP caching + versioned assets

Serve the widget JavaScript and each widget's configuration from **public endpoints** with proper **cache headers** and small payloads, the way a CDN would: a **versioned bundle** for the script (cache long, bust on release) and a short-lived cache for config. A widget that loads slowly is a widget customers remove.

4 Public submission endpoint

teaches: CORS + boundary validation

Visitors on external websites submit the form. Your endpoint must accept **cross-origin** requests (correct **CORS**, including **preflight**), **validate** every field before it touches business logic, reject invalid or oversized payloads with clean 4xx responses, and safely store what passes. The server never trusts the client — here, the client is the entire internet.

5 Protection, enrichment & safe side effects

teaches: abuse resistance + graceful degradation

Three layers run between "request arrives" and "row stored":

- **Abuse protection** — rate limiting per IP and per widget, plus at least one spam control (a **honeypot** field, a heuristic, or a token). A flood of requests must not take you down.
- **Enrichment with a fallback chain** — IP → geolocation, trying provider A, then provider B on failure. If *all* providers are down, the submission still succeeds, just without geo data.
- **Safe side effects** — after storing, trigger a confirmation email / webhook / notification. If that secondary action fails, **the submission itself must still succeed**. Non-critical operations never break the main path.

6 Owner dashboard API

teaches: aggregation queries

The authenticated owner views their submissions with basic analytics: counts over time, per-widget stats, geo breakdown. Endpoints + a simple table are enough — this is a backend capstone, not a frontend one.

5 · Architecture overview

One request path per actor: the owner manages widgets (authenticated), the customer site loads the script (cached, public), the visitor submits (public, CORS, protected). Keep the three paths separate in your head and the code stays clean:

```
Widget Owner (authenticated)
└─ Widget Management API → Widget DB (tenant-isolated) → embed snippet

Customer Website (any origin)
└─ <script src="widget.js?id=123">
    └─ GET /widgets/:id/config (public · cached · CORS)
        └─ render widget

Website Visitor
└─ POST /submissions (public · CORS)
    └─ validation — bad payload? → 4xx, never 500
    └─ rate limit + spam check — flood? → 429, service stays up
    └─ geo enrichment: Provider A -(fails)→ Provider B -(fails)→ store anyway
    └─ store submission
    └─ email / webhook side effect (failure must NOT block success)
```

```
Widget Owner (authenticated)
  ↳ Dashboard API ← submissions + stats
```

6 · Definition of done — the core checklist

This is the contract. **Done** = every box below ticked, with one pasted proof per box in `EVIDENCE.md`. Each box is written so a reviewer can verify it in minutes.

WIDGET MANAGEMENT

- ☐ Authenticated CRUD endpoints for widgets; requests without valid auth are rejected.
- ☐ Multi-tenant isolation proven: tenant A cannot read or modify tenant B's widgets or submissions.
- ☐ Embed snippet generated per widget.

WIDGET DELIVERY

- ☐ Public config endpoint serves a small payload with correct HTTP cache headers.
- ☐ Widget JavaScript is served as a versioned bundle (new version = new URL or cache-bust).
- ☐ The widget renders on a page served from a *different origin* than your API.

PUBLIC SUBMISSION API

- ☐ Cross-origin submissions work: CORS headers correct, preflight (`OPTIONS`) handled.
- ☐ All incoming input validated; malformed and oversized payloads rejected with appropriate 4xx codes and JSON errors.
- ☐ Valid submissions stored safely, linked to the right widget and tenant.

ABUSE PROTECTION

- ☐ Rate limiting per IP and/or per widget returns `429` under a burst — and the API keeps serving legitimate traffic.
- ☐ At least one spam-prevention technique (honeypot field, token, or heuristic) demonstrably blocks a spam submission.

ENRICHMENT & SAFE SIDE EFFECTS

- ☐ IP→geo enrichment uses a provider fallback chain: provider A down → provider B answers → submission enriched.
- ☐ All providers down → submission still succeeds (without geo). Degrade, never fail.
- ☐ A failing confirmation email / webhook does not prevent the submission from being stored.

TESTS & DOCUMENTATION

- ☐ Automated tests cover: CORS preflight, invalid payload, oversized payload, rate limiting, spam control, provider fallback, and successful widget rendering.
- ☐ README with architecture diagram, setup instructions, and API documentation; the five submission-pack files from § 11 present.

7 · Realistic scope — where to stop

Where "enough" is — stop gold-plating past this line:

- **The customer site is a plain HTML file** opened from a different origin (a second local port or `file://`). No hosting, no domain, no real CDN.
- **The widget UI can be minimal** — a div with a form and a submit button. The grade lives in the backend, not the CSS.

- **One or two widget types are plenty** if the model supports more. Prove the pattern, don't build a form-builder startup.
- **Email can be fake:** log it to the console or use a free local catcher (Mailpit). What's graded is that its *failure doesn't block success*.
- **Mock the geo providers in tests** so the fallback is deterministic. Real free APIs are for manual dev only.

8 · The build, phase by phase

This internship is **self-paced** — there is no calendar and no deadline. Work through the phases **in order, at your own speed**: the track assignments are the parts, the capstone assembles them. Each phase ends with a **gate** — a concrete result that tells you it's safe to move on. The effort estimates are just orientation; take what you need. Short on time overall? Shrink scope (§ 7), don't skip phases.

Phase 1 Design · ≈4–6 h

gate: design doc signed off

- Widget model + submission model (fields, tenancy, indexes)
- The embed flow: snippet → config → render → submit
- API contracts for all three request paths
- One explicit non-goal written down

Phase 2 The hardened submission path · ≈14–20 h

gate: cross-origin curl stores an enriched row

- Public submission endpoint: validation + CORS + honest status codes
- Rate limiting + one spam control
- Geo enrichment with the provider fallback chain
- Safe side effect (email/webhook that may fail harmlessly)

Phase 3 Delivery, dashboard & proof · ≈12–16 h

gate: widget renders on a second-origin page; tests green

- Widget script + cached, versioned config delivery
- The "customer site" test page on a different origin
- Dashboard endpoints with stats
- Automated tests + README + **EVIDENCE.md** filled as you go

Phase 4 Demo prep · ≈2–3 h

gate: you've run the § 13 script twice

- Seed demo data, rehearse the demo flow, prepare the one failure you'll show live

9 · Stretch goals — only if the core ships

★ **Optional, and only after every § 6 box is green.** A finished core with one polished stretch beats three half-stretches. Each of these is a genuine "I went deep" interview story:

- **Production bundle:** a minified, versioned widget build (study FlyRank's `cdn/popovers/build.script.js`).
- **Real-time dashboard:** new submissions appear live via WebSockets or Server-Sent Events.
- **Targeting rules:** show on certain pages, after N seconds, or once per visitor.
- **Double opt-in + GDPR:** confirmation flow, consent record, export/delete endpoints.
- **Bot defense:** proof-of-work or CAPTCHA challenge — and measure the spam reduction.

10 · Your \$0 stack — the free-tools promise

We promised you can finish this internship without paying for anything. Here is the proof for this capstone. Every requirement maps to a tool that is free with **no credit card**:

You need	Free tool (no credit card)	Notes
Language + framework	Node.js + Express or Python + FastAPI	Free, as all track long
Database	PostgreSQL via Docker (or SQLite to start)	Free · docker compose up
Geo provider A	ip-api.com (JSON endpoint)	Free, no key, 45 req/min
Geo provider B (fallback)	ipapi.co	Free tier ~1,000 lookups/day, no card
Email side effect	Console log, or Mailpit (local mail catcher)	Free · failure-tolerance is what's graded
The "customer site"	Plain HTML file on a second local port (<code>npx serve / python -m http.server</code>)	Free · that's your second origin
Repo + CI	GitHub (public repo)	Free
Hosting	None required — everything runs locally	Deploying is optional, free tiers only

The iron rule: if any tool, tier, or tutorial asks for a credit card, it is the wrong path — a free alternative for this capstone exists in the table above. Stuck anyway? Ask in the community before paying for *anything*.

11 · GitHub rules — your public repo

Your capstone is also your **portfolio piece**. These rules make the repo something a recruiter, a mentor, and our automated evaluator can all navigate without asking you anything.

The non-negotiables

- ☐ **One dedicated repository, public from day one.** Create it the day you choose your capstone. Do **not** build in your assignments repo, a private repo you flip later, or a monorepo shared with other work. Progress in the open is part of the story — an honest half-built history beats a single "final version" push.
- ☐ **Name it clearly:** `flyrank-capstone-<short-name>` (for this capstone we suggest `flyrank-capstone-widget-platform`). Lowercase, hyphens, no spaces.
- ☐ **Commit as you build.** Small, meaningful commits with messages that say what changed (`Add idempotency key check to publish endpoint`, not `update stuff`). Aim for at least one commit per working session; each phase in § 8 should be visible in your history.
- ☐ **Never commit a secret.** No API keys, tokens, passwords, or `.env` files — put `.env` in `.gitignore` before your first commit and ship a `.env.example` with placeholder values instead. A leaked key in your history means rotating the key and rewriting history — ask for help the moment it happens.
- ☐ **A stranger can run it.** The README's setup section must work on a clean machine with one documented run command (`docker compose up` or equivalent) plus a seed step for demo data.

Required files at submission

File	What goes in it
<code>README.md</code>	What the system does, an architecture diagram (an image or ASCII sketch), exact run + seed steps, and an honest "limitations" note.
<code>capstone.yaml</code>	A small manifest the evaluator reads: <code>run:</code> (one command), <code>seed:</code> , <code>test:</code> , <code>base_url:</code> and the endpoints to probe.
<code>EVIDENCE.md</code>	One pasted proof per Definition-of-Done checkbox in § 6 — a test name + output, a curl transcript, or a log line. Claims without evidence score as not done.
<code>BUILDLOG.md</code>	Your AI-usage log: where AI helped, where it was wrong, what you changed. Honesty is graded, perfection is not.
<code>.env.example</code>	Every environment variable the app needs, with safe placeholder values.

DO

- Create the repo public before you build, first commit = README skeleton + `.gitignore`.
- Add a license (MIT is a fine default) so others may read and learn from it.
- Use branches if you like — but keep `main` always runnable.
- Paste real command output into `EVIDENCE.md` as you finish each checkbox — not in a panic at the end.

DON'T

- Don't mix capstone code with track assignments — separate repos, always.
- Don't commit `node_modules/`, `virtualenvs`, or datasets over a few MB.
- Don't force-push over your history before submission — the journey is evidence.
- Don't commit real tokens "just for a second". There is no such thing on a public repo.

12 · How your capstone is evaluated

Three layers, published up front — **you know exactly what will be checked, so build to pass it.**

Layer 1 — The submission pack (machine-checkable)

The portal first checks your repo structure: the five required files from § 11, a `run:` command that boots the system, and a `test:` command that runs green. Missing pack files cost points before a human ever looks.

Layer 2 — Acceptance probes (behavioral, pass/fail)

An evaluator (human or automated) runs these against your live system. They are not secrets — they are promises:

PROBE 1 — POST a valid submission from the second-origin test page → stored, `2xx`, and visible via the dashboard API.

PROBE 2 — Send a malformed and an oversized payload → clean `4xx` JSON errors, never a `500`.

PROBE 3 — Fire a burst of rapid submissions → `429`s appear, and a normal request right after still succeeds.

PROBE 4 — Disable geo provider A → the next submission is stored, enriched by provider B. Disable both → stored anyway, without geo.

PROBE 5 — Force the email/webhook side effect to throw → the submission still returns success and is stored.

PROBE 6 — Fill the honeypot field like a bot would → the submission is silently dropped or rejected.

Layer 3 — The rubric (judgment, 1–5 per dimension)

The weights say what we value: **correctness and resilience over feature count.**

Dimension	What a "5" looks like	Weight
Architecture	Clean layers; swap the DB or a provider without touching business logic	×3
Correctness	Happy path solid; edge cases handled; probes all pass	×3
Resilience	Survives bad input, a dead dependency, a job running twice	×3
Security	Real authorization; validated input; secrets encrypted & never logged	×2
AI cost & grounding (if AI used)	Cost metered + capped; outputs grounded; eval catches regressions	×2
Testing	The scary cases, deterministically; evals for AI behavior	×2
Communication	README + diagram tell the story; honest about limits; demo lands	×2

Levels: **Ships** = every core box in § 6 done + all probes pass (a real pass) · **Solid** = Ships + rubric average ≥ 3.5 · **Exceptional** = Solid + meaningful stretch goals. A small system that is correct, resilient and well-tested beats a huge one that falls over — that is what senior engineers actually value.

The shared requirements (every capstone must show these)

These eight patterns are how we know you absorbed the program, not just this topic. You built each one in an assignment already:

#	Requirement	Where you learned it
---	-------------	----------------------

1	Layered architecture — data / logic / HTTP separated	A2 + the architecture live event
2	Validation at the boundary — bad input → clean 4xx, never a 500	A5
3	≥1 background job — slow/bulk work off the request path, retries + failure alert	A7
4	Real persistence — schema as migrations, right indexes, isolated tenants	A5
5	Idempotency where it matters — the retried action happens once	A5 stretch + A10
6	Secrets clean — env only, encrypted if stored, never logged	A10
7	Cost tracked, if AI is used — per call, attributed, with a budget guard	A6
8	Tests that matter — the scary cases, deterministic; AI features get an eval	A13

13 · The final demo — your 6 minutes

Rehearse this flow start to finish. A demo that shows **one failure handled gracefully** impresses more than ten happy paths.

1. Create a widget through your authenticated API; show the generated `<script>` snippet.
2. Open the plain HTML "customer site" on a different origin — **watch your widget appear on a page you didn't build**.
3. Submit the form; show the submission landing in the dashboard, enriched with location data.
4. Now attack yourself: send an invalid payload (rejected), a cross-origin request with a disallowed origin (CORS behaves), a burst (rate limiter answers `429`).
5. Kill the primary geo provider live — the fallback provider takes over mid-demo.
6. Break the email side effect — the submission still succeeds. Say the sentence: *"non-critical failures never break the main path."*
7. Close on the dashboard: submissions, stats, geo — **your application safely accepts data from websites you don't own**.

Study these FlyRank parts first

FlyRank runs this exact pattern in production. Before building, read (in the shared program codebase excerpts):

FlyRank reference	What to steal from it
<code>cdn/popovers/</code> — <code>embed.script.ts</code> , <code>popover.script.ts</code> , <code>build.script.js</code>	How a one-line embed bootstraps config fetch + render; how the bundle is built and versioned
<code>app/api/popover-submissions/route.ts</code>	The real public submission endpoint: validation, 3-provider geo fallback, safe email side effect, CORS
<code>services/popovers.service.ts</code> · <code>repositories/popovers.repository.ts</code>	Widget config CRUD through clean layers

14 · Curated resources — free, verified, leveled

Don't read everything. Each row says when to reach for it. Every link was verified while writing this brief; every resource is free with no credit card. If a link ever dies, the title is searchable.

● **Start here** (everyone) · ● **When you're comfortable** · ● **Optional deep dive** | Lanes: **JS** · **PY** · **both**

Phase 1 · Design

Lane	Resource	Format	When to use it
● both	MDN — Cross-Origin Resource Sharing (CORS)	Article, ~30 min	Read before designing the public endpoint — origins, preflights, headers. Your future debugging map.
● both	Cloudflare — What is rate limiting?	Article, ~10 min	Read while sketching abuse protection.
● both	How to create a simple form honeypot	Article, ~10 min	Read before designing the spam-filter step.
● both	MDN — HTTP caching guide	Article, ~25 min	When planning how the versioned bundle and config are served.

Phase 2 · The hardened submission path

Lane	Resource	Format	When to use it
● JS	Zod — Basic usage	Docs, ~15 min	Before writing payload validation in Express.
● PY	Pydantic — Models	Docs, ~25 min	Before writing payload validation in FastAPI.
● JS	Express cors middleware	Docs, ~15 min	Open while wiring CORS onto the submission route.
● PY	FastAPI — CORSMiddleware	Docs, ~10 min	Open while adding CORS in FastAPI.
● JS	express-rate-limit — Overview	Docs, ~15 min	When adding per-IP limits (PY lane: slowapi works the same way).
● both	ip-api.com — JSON API docs	Docs, ~10 min	When building IP→geo enrichment. No key, no card, 45 req/min.
● both	ipapi.co — IP geolocation API	Docs, ~10 min	Your second provider for the fallback chain (~1,000 lookups/day free).
● both	MDN — HTTP status codes	Reference, ~10 min	Keep open — pick honest codes for 400/401/404/413/429.

Phase 3 · Delivery & hardening

Lane	Resource	Format	When to use it
● both	Building an embeddable JavaScript widget	Article, ~20 min	Follow along before building the embed script and loader.
● both	MDN — Cache-Control reference	Reference, ~15 min	When tuning <code>max-age</code> , <code>immutable</code> , and cache-busting for the bundle.

15 · Glossary

Plain-language definitions of the **bold** terms in this brief. No definition depends on another — read in any order.

Term	What it means
Origin	The scheme + domain + port a page is served from. <code>http://localhost:3000</code> and <code>http://localhost:5500</code> are two different origins — which is exactly how you'll test.
CORS	Cross-Origin Resource Sharing — the browser rules deciding whether a page from one origin may call an API on another. Your submission endpoint must explicitly allow it.
Preflight	The automatic <code>OPTIONS</code> request a browser sends before certain cross-origin calls, asking "am I allowed?". Your API must answer it correctly or the real request never arrives.
Embed snippet	The one-line <code><script></code> tag a customer pastes into their website to load your widget.
Versioned bundle	A JavaScript file whose URL changes when its content changes (e.g. <code>widget.v2.js</code>), so browsers can cache it forever without serving stale code.
Cache headers	HTTP response headers (like <code>Cache-Control</code>) telling browsers and proxies how long they may reuse a response without asking again.
Rate limiting	Refusing requests above a threshold (per IP, per widget) with <code>429</code> , so a flood can't take the service down.
Honeypot	A hidden form field humans never fill but bots do. A filled honeypot = spam, rejected without telling the bot why.
Enrichment	Adding derived data to a submission before storing it — here, turning the visitor's IP address into country/city.
Fallback chain	Trying providers in order (A, then B) so one dead upstream doesn't kill the feature. The pattern behind "degrade gracefully".
Side effect	A secondary action after the main work (confirmation email, webhook). Safe side effects can fail without breaking the main response.
Tenant isolation	Guaranteeing each customer (tenant) can only ever see their own data, enforced in every query — not just in the UI.
Payload	The body of a request — here, the submitted form data your API must validate before trusting.

FlyRank Internship · Backend Development Track · Capstone — Embeddable Widget & Lead-Capture Platform. Everything in this brief can be completed with free tools; no resource linked here requires a credit card. Questions → the capstone channel on the community. Version: July 2026.